GITHUB · IA AGÉNTICA

Gobernanza de sistemas agénticos en GitHub

Autonomía proporcional al riesgo: límites explícitos, verificables y auditables

A mayor riesgo, menor autonomía.

Bajo Medio Alto Crítico



LA CUESTIÓN FUNDAMENTAL

Del «qué puede hacer» al «qué debe poder hacer»

Lo que un agente ya puede hacer

- Analizar repositorios
- Proponer soluciones
- Modificar archivos
- Crear ramas y Pull Requests
- Ejecutar validaciones
- Iterar con los resultados

Lo que debería estar autorizado a hacer

Explícitos
Límites definidos de antemano

Verificables
Checks y políticas

Auditables
Evidencia de lo ocurrido



A mayor riesgo, menor autonomía. A menor riesgo y mayor reversibilidad, mayor posibilidad de automatización.

01 · AUTONOMÍA BASADA EN RIESGO

No todas las acciones tienen las mismas consecuencias

Documentación es fácil de revertir; infraestructura, workflows, secretos o producción pueden afectar a todo el sistema.

AUTONOMÍA	EJEMPLOS	CONTROL
Bajo ●●●●	Documentación, formato	**Automatización**
Medio ●●●○	Dependencias, refactors seguros	**PR + checks**
Alto ●●○○	Infraestructura, workflows	**CODEOWNERS + approvals + checks**
Crítico ●○○○	Producción, secretos	**Environment + aprobación humana + evidencia**



Alto riesgo por defecto

`.github/workflows/`

`infra/`

`security/`

Diff pequeño, consecuencias potencialmente grandes.

02 · NIVELES DE AUTONOMÍA

Cuatro niveles para diseñar agentes

1



Solo lectura

El agente puede:

- Inspeccionar
- Analizar
- Clasificar
- Resumir
- Recomendar

No modifica recursos

2



Solo propuesta

El agente puede:

- Crear branches
- Realizar cambios
- Abrir Pull Requests

Sin merge ni deploy

3



Ejecución con guardrails

Operaciones previamente aprobadas:

- Tests
- Builds
- Análisis de seguridad
- Deployments a staging

Dentro de límites

4



Ejecución autorizada por humanos

Aprobación explícita para:

- Deploy a producción
- Modificar workflows
- Cambios de infra
- Acceso a secretos

Decisión humana

PIPELINE · EJECUTAR SEGÚN EL RIESGO

El riesgo como dato: el workflow decide el camino



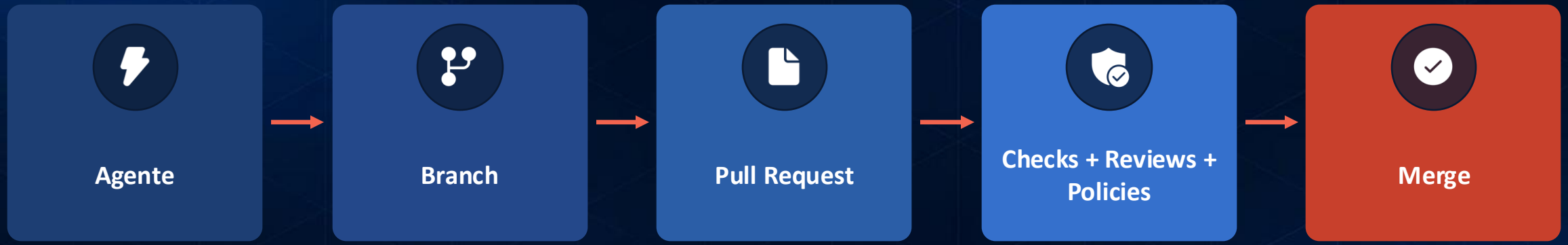
agent-plan-apply.yml (extracto)

```yaml
plan:
  outputs:
    risk: ${{ steps.read.outputs.risk }}
  steps:
    - run: |
        echo "risk=$(jq -r .risk out/plan.json)" \
          >> "$GITHUB_OUTPUT"

apply:                # medio / alto
  needs: plan
  environment: approval-required
  if: ${{ needs.plan.outputs.risk != 'low' }}
  steps:
    - run: ./scripts/apply.sh out/plan.json

apply_auto:           # bajo
  needs: plan
  if: ${{ needs.plan.outputs.risk == 'low' }}
  steps:
    - run: ./scripts/apply.sh out/plan.json
```

El Pull Request como frontera de control



El agente no trabaja directamente sobre main

Proponer y ejecutar no significa decidir.

El Pull Request permite aplicar:

- Required checks
- Required reviews
- CODEOWNERS
- Rulesets
- Branch protection
- Análisis de seguridad
- Aprobación humana

Una rama protegida exige, no solo recomienda

Protección de main

- Pull Request para integrar cambios
- Required checks exitosos
- Approving reviews
- CODEOWNERS para áreas sensibles
- Restricción del push directo
- Bloqueo de force pushes
- Bloqueo de eliminación de la rama

Checks ligados a riesgos concretos: build, unit tests, lint, security scanning, políticas y controles para infra/ y .github/workflows/

CODEOWNERS

```
/security/          @security-team
/infra/             @platform-team
/.github/workflows/ @platform-team
*                   @core-team
```

Tener un CODEOWNERS no es suficiente

La configuración del repositorio debe exigir las reviews correspondientes, para que cambios en infra o workflows involucren a los equipos responsables.

05 · HUMAN IN THE LOOP

Humanos en los puntos de decisión de alto impacto

1. **Agente crea el PR** — AUTOMÁTICO
2. **Checks automáticos** — AUTOMÁTICO
3. **CODEOWNERS / reviewers** — HUMANO
4. **Aprobación** — HUMANO
5. **Merge** — HUMANO

Decisiones que requieren una persona

Se conserva la velocidad sin renunciar al control.

- **Merge**
- **Deployment**
- **Acceso a secretos**
- **Cambios de infraestructura**
- **Modificación de controles de seguridad**

Aprobación humana y deployments serializados

Aprobación para producción

GitHub Environment

```yaml
jobs:
  deploy:
    runs-on: ubuntu-latest
    environment:
      name: production
    steps:
      - run: ./deploy.sh
```

El agente prepara el deployment y ejecuta las validaciones previas; la autorización final permanece en manos de los responsables del entorno.

Evitar deployments simultáneos

Concurrency

```yaml
concurrency:
  group: production
  cancel-in-progress: true
```

LA SERIALIZACIÓN AYUDA A EVITAR

- Race conditions
- Releases incompatibles
- Estados inconsistentes
- Ejecuciones difíciles de auditar

07 · DEFENSE IN DEPTH

Los controles no dependen de una sola barrera

01

Pre-action constraints

Antes de actuar

- Permisos
- Tool allowlists
- Hooks

02

In-action validation

Durante la acción

- Tests
- Security scans
- Policy checks

03

Post-action traceability

Después de actuar

- Logs
- Artifacts
- Approvals
- Historial del PR

Una instrucción al agente no equivale a un control técnico.

Decir "no modifiques producción" orienta al modelo. **Un Environment protegido, permissions o un hook pueden impedir técnicamente la operación.**

PIPELINE · GUARDRAILS

Un segundo workflow verifica antes del merge

agent-guardrails.yml (extracto)

```yaml
on:
  pull_request:
    branches: ["main"]

permissions:
  contents: read

jobs:
  enforce-human-review:
    steps:
      - name: Fail if PR is authored by Copilot agent
        if: ${{ github.event.pull_request.user.login
                == 'github-copilot[bot]' }}
        run: exit 1

      - name: Require at least one human approval
        uses: actions/github-script@v7
        # pulls.listReviews -> APPROVED y user.type == "User"
        # core.setFailed("Human approval is required")
```

Bloquea PRs del bot

Los PRs de github-copilot[bot] deben ser abiertos o poseídos por una persona.

Exige aprobación humana

Al menos una review APPROVED de un usuario de tipo User.

Defensa en profundidad

No reemplaza branch protection, rulesets ni CODEOWNERS.

Permisos mínimos y secretos bajo control

permissions por job

```yaml
permissions:
  contents: read          # por defecto

jobs:
  analyze:
    steps:
      - run: echo "Read-only analysis"

  update_artifacts:
    permissions:
      contents: write
      pull-requests: write
    steps:
      - run: echo "Write operations happen here"
```

Read-only por defecto; elevar solo cuando haga falta.

Reduce el blast radius de un fallo o acción inesperada.

NUNCA ESCRITOS EN

- Código fuente
- Instrucciones del agente
- Archivos versionados
- YAML del workflow en texto plano

DEBEN

- Inyectarse durante runtime
- Existir solo donde sean necesarios
- Limitarse por Environment si corresponde
- Seguir el mínimo privilegio

El acceso a secretos de producción es una operación crítica.

Sin evidencia, no hay validación suficiente

Debemos poder responder

1. ¿Qué intentó hacer el agente?
2. ¿Qué modificó?
3. ¿Qué validaciones se ejecutaron?
4. ¿Cuál fue el resultado?
5. ¿Quién aprobó el cambio?

Fuentes de evidencia en GitHub

Pull Requests	Commits	Workflow runs
Job logs	Required checks	Security scans
Reviews	Approvals	Artifacts

upload-artifact

```yaml
- name: Upload execution report
  uses: actions/upload-artifact@v4
  with:
    name: execution-report
    path: report.json
```

EVIDENCIA CRÍTICA

```
plan.json
test-results/
security-report/
execution-report.json
```

Si un cambio no puede auditarse, no es una ejecución confiable.

La autonomía también depende de las herramientas

Copilot CLI · solo analizar el historial Git

```
copilot -p "Summarize the last 10 commits and highlight breaking changes." \
  --allow-tool 'shell(git)'
```

Copilot CLI · solo leer y buscar

```
copilot agent run CodeAgent --allow-tool 'read,search'
```



Las capacidades disponibles deben corresponder a las consecuencias admitidas.

Un agente que solo necesita analizar el historial Git no debería recibir permisos generales sobre la terminal.

La autonomía crece a medida que se gana confianza

No es necesario comenzar con autonomía máxima.

FASE 1



Abre PRs, pero no hace merge.

FASE 2



Integra cambios de bajo riesgo después de los checks.

FASE 3



Despliega automáticamente a staging.

FASE 4



Prepara producción; el Environment exige aprobación humana.

La gobernanza no termina tras la configuración

Así se degradan los controles

- Checks eliminados o renombrados
- Permisos ampliados
- CODEOWNERS desactualizado
- Approvals relajadas
- Secretos movidos a scopes más amplios
- Nuevos workflows sin los guardrails originales

Semanalmente

- Revisar workflows fallidos
- Analizar violaciones frecuentes de políticas

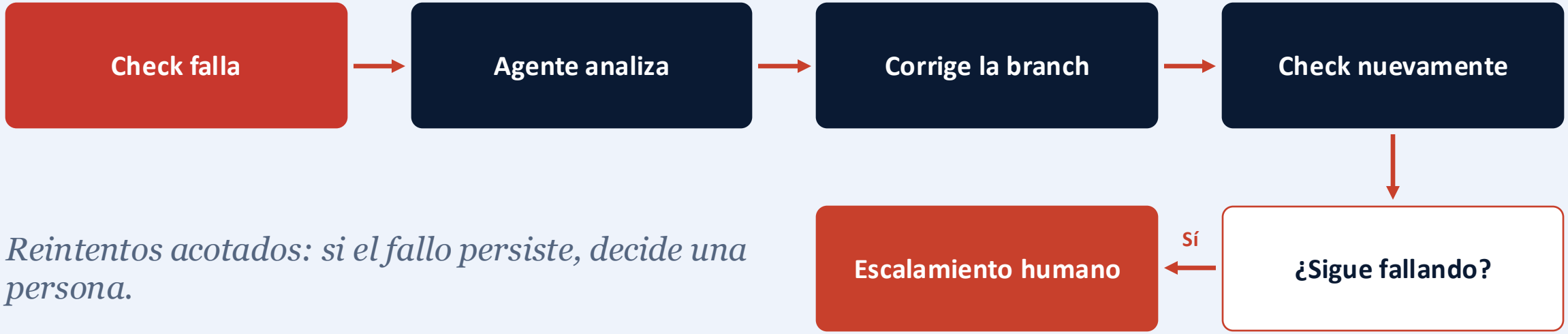
Mensualmente

- Revisar permisos
- Revisar scopes de secretos

Trimestralmente

- Auditar Rulesets, CODEOWNERS y Environment reviewers
- Políticas de evidencia y retención

Diseñar suponiendo que habrá fallos



Reintentos acotados: si el fallo persiste, decide una persona.

- Retries acotados para errores transitorios
- Escalamiento tras fallos repetidos
- Pull Requests pequeños
- Mecanismos simples de revert
- Rollback explícito para operaciones de alto impacto
- Investigar fallos de seguridad en lugar de reintentarlos

Tres preocupaciones, un recorrido hasta el deploy

Planning

¿Qué debe hacerse y por qué?

1. Tarea
2. Planning Agent
3. Plan + clasificación de riesgo
4. Aprobación cuando corresponda

Execution

¿Qué cambios concretos se realizaron?

5. Implementation Agent
6. Branch
7. Pull Request

Validation

¿Qué evidencia demuestra que el resultado es correcto y seguro?

8. Tests + Security + CODEOWNERS
9. Required Checks
10. Human Review
11. Merge
12. Environment Gate
13. Deploy

PRINCIPIOS FINALES

Diez principios para agentes gobernables

01

No trabajar directamente sobre main

02

Hacer visible el plan del agente

03

Separar Planning, Execution y Validation

04

Una instrucción no es un control técnico

05

Definir criterios de éxito verificables

06

Registrar evidencia de cada ejecución

07

Ajustar la autonomía al nivel de riesgo

08

Diseñar suponiendo que habrá fallos

09

Aplicar least privilege en agentes, workflows y herramientas

10

Mantener decisiones críticas bajo controles humanos y políticas exigibles

IDEA FUNDAMENTAL

Un agente confiable no es el que tiene más autonomía, sino el que puede actuar dentro de límites explícitos, verificables, auditables y proporcionales al riesgo.

GitHub permite construir estos límites combinando:

- Pull Requests
- GitHub Actions
- Required Checks
- Rulesets
- Branch Protection
- CODEOWNERS
- Environments
- Secrets
- Artifacts
- Hooks



El objetivo no es eliminar la autonomía, sino hacerla ***segura, observable, reversible y gobernable.***